

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas
Manejo e Implementación de Archivos

Manual técnico de Sistema de archivos EXT2 simulado
Proyecto 1

Sophía Raquel Pérez Marroquin

202405047

Guatemala, 13 de septiembre de 2026

1 Introducción y objetivo

ExtreamFS es una aplicación web para estudiar la administración de discos y un sistema de archivos EXT2 simulado. El usuario escribe comandos en una interfaz React; el backend C++ los procesa mediante una API REST y opera sobre archivos binarios con extensión .mia. Cada archivo .mia representa un disco de tamaño fijo, dentro del cual se organizan particiones, metadatos y contenido.

La idea central del sistema es que una ruta, como /docs/a.txt, se resuelve recorriendo estructuras almacenadas en el disco simulado. El nombre del archivo está en una entrada de carpeta; sus permisos y ubicación de contenido están en un inodo; sus bytes se almacenan en bloques. Esta separación permite explicar la creación, lectura y administración de archivos sin depender de un disco físico dedicado.

1.1 Objetivo del manual

Describir la arquitectura, la organización binaria y los comandos de ExtreamFS para facilitar su comprensión, mantenimiento y defensa oral. Al finalizar la lectura, el estudiante debe poder explicar qué componente atiende una solicitud, qué estructuras consulta o modifica y cómo comprobar el resultado mediante reportes.

1.2 Alcance técnico

El alcance comprende los 17 comandos de administración de discos, particiones, sesiones, usuarios, grupos, archivos, carpetas y reportes detallados en este manual. La simulación utiliza las estructuras definidas en el enunciado del Proyecto 1; no equivale a una imagen EXT2 de producción que pueda montarse directamente con las herramientas del sistema operativo.

El archivo .mia conserva el tamaño asignado por MKDISK. Los archivos y carpetas deben gestionarse desde sus estructuras persistentes, sin mantener árboles o listas en RAM que sustituyan su representación binaria. El registro temporal de montajes y la sesión tienen una función distinta: identificar la partición y el usuario que operan actualmente.

1.3 Organización del contenido

Secciones	Contenido
2	Arquitectura y recorrido de una solicitud.
3 y 4	Estructuras fundamentales y distribución física.
5	Permisos UGO y administración de identidad.
6	Referencia de los 17 comandos.
7 y 8	Scripts .smia y reportes.
9 y 10	Conclusión técnica y referencia documental.

2 Arquitectura del sistema

ExtreamFS separa la presentación, la recepción de solicitudes, el análisis de comandos y el acceso a los datos. React representa la capa de interfaz indicada para esta aplicación. El enunciado permite elegir el framework del frontend y exige que la lógica del backend se desarrolle en C++ y se comunique mediante APIs RESTful.

2.1 Frontend React

La interfaz contiene un área de entrada, un botón para cargar scripts, un botón para ejecutar y un área de salida. React administra el texto que está editando el usuario, el contenido cargado desde un .smia y los mensajes recibidos. Al ejecutar, entrega el texto al backend por HTTP y posteriormente muestra resultados, errores y reportes disponibles.

El frontend no decide en qué byte se escribe un inodo ni modifica directamente el .mia. Las rutas usadas para crear discos o importar contenido se interpretan en el entorno al que tiene acceso el backend. En una ejecución local ambos componentes pueden estar en la misma computadora, pero siguen teniendo responsabilidades separadas.

2.2 Backend C++ y API REST

La API es el punto de entrada al servidor. Recibe la solicitud, entrega el texto al analizador y devuelve una respuesta que la interfaz pueda presentar. Su contrato debe transportar comandos y resultados de manera consistente. El PDF no establece nombres concretos de endpoints, bibliotecas HTTP ni un esquema obligatorio de mensajes; estos pertenecen a la implementación.

El analizador identifica el comando y sus parámetros. El análisis léxico reconoce palabras, guiones, signos igual, comillas y valores; el sintáctico comprueba su organización. Después, la validación del comando comprueba los parámetros admitidos, los obligatorios, sus valores y las condiciones de la operación. Un nombre desconocido, una ruta inexistente y la falta de espacio son errores de naturaleza distinta y deben explicarse con mensajes claros.

2.3 Comandos y managers

La capa de comandos expresa la intención de la operación y coordina las validaciones. Los managers son módulos de servicio organizados por responsabilidad: discos y particiones, montajes, sistema de archivos, usuarios y reportes. Esta división es una descripción funcional de la arquitectura; no implica agregar estructuras persistentes al formato del proyecto.

Los managers calculan posiciones, leen las estructuras necesarias y escriben los cambios en el archivo binario. Por ejemplo, el servicio de archivos recorre carpetas e inodos, mientras que el servicio de usuarios modifica el contenido de /users.txt mediante los mecanismos del sistema de archivos.

2.4 Flujo de procesamiento

El siguiente diagrama resume el recorrido de una operación. Las flechas representan el orden de trabajo; la respuesta reúne el resultado o el error producido y regresa al área de salida de React.

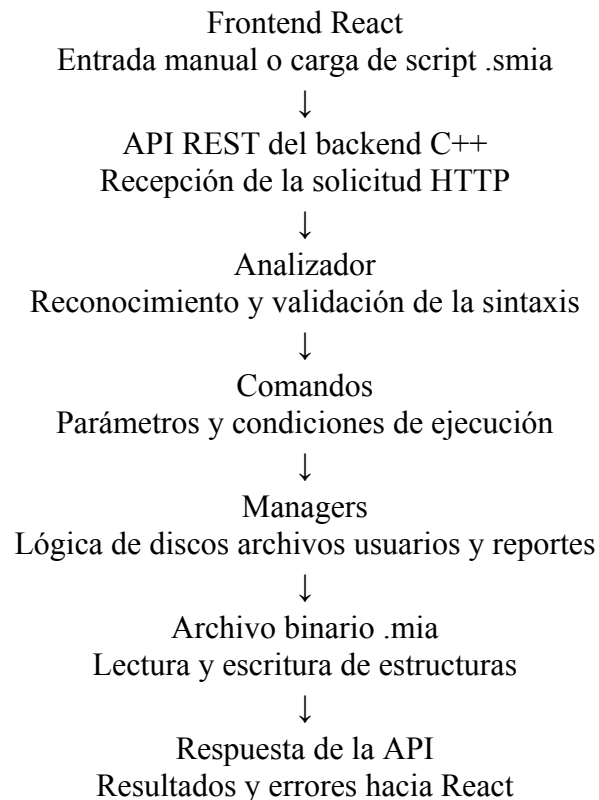


Figura 1. Flujo general de ExtreamFS. MOUNTED y LOGOUT trabajan principalmente con estado temporal; no todas las solicitudes requieren escribir en el .mia.

2.5 Ejemplo de recorrido de MKFILE

Al recibir `mkfile -path=/docs/a.txt -size=15`, el analizador identifica MKFILE y sus dos parámetros. La capa del comando comprueba la sesión, el tamaño y la ruta. El manager localiza `/docs`, verifica escritura y consulta los bitmaps. Si hay espacio, asigna un inodo y los bloques necesarios, registra `a.txt` en la carpeta padre y actualiza los contadores. El backend devuelve el resultado y React lo muestra.

Si una validación falla, se comunica la causa. Si el archivo ya existe, el flujo debe solicitar la decisión de sobrescritura indicada por el enunciado antes de sustituir su contenido. La existencia de un mensaje de éxito debe corresponder con el estado que luego se lee desde el .mia.

3 Estructuras de datos fundamentales

Los nombres MBR, Partition, EBR, SuperBlock, Inode, Content, FolderBlock, FileBlock y PointerBlock identifican las estructuras descritas por el PDF. En sus tablas, el documento utiliza también los términos súper bloque, inodos y bloques. Se conservan los atributos definidos; los bitmaps son regiones de control de ocupación.

3.1 MBR

El Master Boot Record se escribe al inicio del .mia. Describe el disco completo y contiene cuatro descriptores Partition. Se consulta para localizar las particiones; no contiene por sí mismo los archivos del usuario.

Atributo	Tipo	Función
mbr_tamano	int	Tamaño total del disco en bytes.
mbr_fecha_creacion	time	Fecha y hora de creación.
mbr_dsk_signature	int	Identificador aleatorio del disco.
dsk_fit	char	Ajuste del disco: B, F o W.
mbr_partitions	Partition[4]	Cuatro entradas para primarias o extendida.

3.2 Partition

Cada entrada describe una región reservada del disco. Crear una primaria actualiza su descriptor en el MBR y reserva un intervalo; el descriptor no se duplica dentro del área de datos de la partición.

Atributo	Tipo	Función
part_status	char	Estado de montaje.
part_type	char	P para primaria o E para extendida.
part_fit	char	Ajuste B, F o W.
part_start	int	Byte donde comienza la partición.
part_s	int	Tamaño de la partición en bytes.
part_name	char[16]	Nombre de la partición.
part_correlative	int	Inicialmente -1; correlativo al montar.
part_id	char[4]	ID generado al montar.

3.3 EBR

El Extended Boot Record describe una partición lógica dentro de una extendida. Al crear la extendida se escribe su primer EBR. Los registros posteriores se relacionan mediante `part_next`, por lo que es posible recorrerlos leyendo un registro a la vez desde el archivo.

Atributo	Tipo	Función
<code>part_mount</code>	<code>char</code>	Estado de montaje de la lógica.
<code>part_fit</code>	<code>char</code>	Ajuste B, F o W.
<code>part_start</code>	<code>int</code>	Byte de inicio indicado para la partición.
<code>part_s</code>	<code>int</code>	Tamaño en bytes.
<code>part_next</code>	<code>int</code>	Byte del siguiente EBR; -1 al final.
<code>part_name</code>	<code>char[16]</code>	Nombre de la partición lógica.

La extendida ocupa una de las cuatro entradas del MBR; las lógicas se describen mediante EBR dentro de ella. No se debe contabilizar una lógica como una quinta entrada Partition. El recorrido debe respetar el intervalo de la extendida y reservar espacio tanto para los descriptores como para las regiones lógicas.

3.4 Relación entre disco y particiones

MBR -> Partition primaria -> sistema de archivos EXT2. Para una extendida, la relación es MBR -> Partition extendida -> EBR -> siguiente EBR. El segundo recorrido organiza particiones lógicas; no sustituye la tabla de inodos de un sistema de archivos.

El PDF restringe el manejo de archivos a particiones primarias. Aunque una observación general menciona lógicas para montaje y reportes, la sección específica de MOUNT limita los montajes de esta simulación a primarias. Por ello, el flujo de formateo y archivos del manual utiliza particiones primarias.

3.5 Tamaños y escritura binaria

Los tamaños de las estructuras se obtienen con `sizeof` en la implementación C++. El tamaño de `int`, la representación de `time` y el relleno que introduce el compilador pueden afectar el tamaño total. No debe deducirse el tamaño de MBR o Inode sumando campos sin comprobar su representación real.

Los arreglos `char` de tamaño fijo no son cadenas de longitud ilimitada. Su lectura y escritura debe respetar la capacidad especificada sin invadir el atributo siguiente. Los bloques del proyecto tienen 64 bytes; esta condición debe verificarse al definir las estructuras C++.

3.6 SuperBlock

El SuperBlock se ubica al comienzo de la región EXT2 de la partición primaria. Es una estructura única cuya información se actualiza durante las operaciones. Describe capacidad, ocupación, fechas y posiciones de los componentes del sistema de archivos.

Atributo	Tipo	Función
s_filesystem_type	int	Identificador del sistema: 2.
s_inodes_count	int	Total de inodos.
s_blocks_count	int	Total de bloques.
s_free_blocks_count	int	Cantidad de bloques libres.
s_free_inodes_count	int	Cantidad de inodos libres.
s_mtime	time	Última fecha de montaje.
s_umtime	time	Última fecha de desmontaje.
s_mnt_count	int	Cantidad de montajes.
s_magic	int	Marca del sistema: 0xEF53.
s_inode_s	int	Tamaño de un inodo.
s_block_s	int	Tamaño de un bloque.
s_firts_ino	int	Primer inodo libre; el PDF lo describe como dirección.
s_first_blo	int	Primer bloque libre; descrito como dirección.
s_bm_inode_start	int	Inicio del bitmap de inodos.
s_bm_block_start	int	Inicio del bitmap de bloques.
s_inode_start	int	Inicio de la tabla de inodos.
s_block_start	int	Inicio de la región de bloques.

Se conserva la escritura s_firts_ino utilizada por el PDF. Los campos de inicio permiten localizar regiones mediante desplazamientos; los contadores libres deben coincidir con los bitmaps. Reservar un bloque modifica su ocupación y reduce el contador correspondiente. Un campo de desmontaje no implica la existencia de un comando adicional en el catálogo del Proyecto 1.

3.7 Inode

Cada archivo o carpeta tiene un inodo que almacena metadatos y referencias a bloques. El nombre se guarda en la entrada de la carpeta que lo contiene. Así, buscar a.txt significa localizar su entrada, obtener su inodo y seguir las referencias a su contenido.

Atributo	Tipo	Función
i_uid	int	UID del propietario.
i_gid	int	GID del grupo propietario.
i_s	int	Tamaño del archivo en bytes.
i_atime	time	Última lectura sin modificación.
i_ctime	time	Fecha de creación.
i_mtime	time	Última modificación.
i_block	15 enteros	12 referencias directas y 3 indirectas.
i_type	char	0 para carpeta y 1 para archivo.
i_perm	char[3]	Permisos UGO en forma octal.

Referencias directas e indirectas

Las posiciones 0 a 11 de i_block son directas. La posición 12 conduce a un bloque de apuntadores simples; la 13, a un recorrido doble; y la 14, a un recorrido triple. Una referencia no utilizada vale -1. Un número de bloque se transforma en una posición física usando la base de la región de bloques y el tamaño del bloque.

En un archivo, una referencia directa alcanza un FileBlock. La indirección simple pasa por un PointerBlock; la doble pasa por dos niveles; la triple pasa por tres. Los PointerBlock también consumen espacio y deben marcarse como ocupados. El PDF describe estos bloques indirectos para inodos de archivo.

Con 16 referencias por PointerBlock, la capacidad teórica de datos por inodo de archivo es $(12 + 16 + 16^2 + 16^3) \times 64 = 280\,320$ bytes. Este valor se deriva de la estructura; la capacidad disponible real puede ser menor por el espacio de la partición y por los bloques de apuntadores necesarios.

Para una carpeta, la relación normal es Inode -> FolderBlock -> Content -> Inode del hijo. La entrada Content es la que asocia el nombre con el inodo hijo; no se confunde una referencia de bloque con una referencia de inodo.

3.8 Content y tipos de bloques

Content y FolderBlock

Content es una entrada de directorio. FolderBlock agrupa cuatro entradas Content. Cada una relaciona un nombre con un inodo. En el primer bloque de una carpeta, los dos primeros registros se reservan para la propia carpeta y su padre, según el enunciado.

Estructura	Atributo y tipo	Función
Content	b_name: char[12]	Nombre del archivo o carpeta.
Content	b_inodo: int	Referencia al inodo asociado.
FolderBlock	b_content: Content[4]	Cuatro entradas de directorio.
FileBlock	b_content: char[64]	Bytes del contenido de un archivo.
PointerBlock	b_pointers: int[16]	Referencias para los niveles indirectos.

Si int ocupa 4 bytes, cada Content ocupa $12 + 4 = 16$ bytes y un FolderBlock ocupa $4 \times (12 + 4) = 64$ bytes. Esta suma expresa el tamaño coherente con los campos; la multiplicación impresa en el PDF contiene un error aritmético.

FileBlock

Almacena hasta 64 bytes del archivo. Un texto de 150 bytes necesita tres bloques de datos: $64 + 64 + 22$. El inodo conserva $i_s = 150$ para distinguir el contenido válido del espacio restante. Los bloques no tienen que quedar físicamente contiguos, porque el inodo y los apuntadores determinan el orden de lectura.

PointerBlock

Contiene 16 enteros y ocupa 64 bytes bajo la representación de int de 4 bytes prevista por el proyecto. Sus entradas conducen a bloques de datos o a otros bloques de apuntadores, según el nivel. No contiene texto del archivo ni nombres de carpetas.

3.9 Bitmaps

El bitmap de inodos y el bitmap de bloques indican ocupación: 0 significa libre y 1 ocupado. El bitmap de bloques no distingue tipos; para interpretar un bloque se considera el inodo y el recorrido que lo referencia. En la fórmula de distribución se reserva un byte por entrada, de modo que hay n bytes para inodos y $3n$ bytes para bloques.

Al crear un objeto se localiza una entrada libre, se marca su ocupación y se actualizan los contadores del SuperBlock. Los datos reservados, sus referencias y los bitmaps deben concordar. El ejemplo aislado del PDF con 10 inodos y 20 bloques ilustra ocupación; la fórmula normativa establece 30 bloques para 10 inodos.

4 Distribución física del disco y EXT2

4.1 Layout del disco

Desde el byte 0 se encuentra el MBR. Después se distribuyen las regiones de particiones y los huecos libres. La suma de primarias y extendidas no supera cuatro; solo existe una extendida y las lógicas permanecen dentro de sus límites. Los intervalos reservados no deben superponerse ni rebasar mbr_tamano.

[MBR] -> [Primaria] -> [Espacio libre] -> [Extendida]
Dentro de la extendida: [EBR] -> [Lógica] -> [EBR] -> [Lógica]

Figura 2. Ejemplo conceptual de distribución. El orden y tamaño de las regiones dependen de las particiones creadas; las flechas indican continuidad visual, mientras part_next enlaza específicamente los EBR.

First Fit selecciona el primer hueco suficiente; Best Fit, el menor hueco suficiente; Worst Fit, el mayor. El MBR registra el ajuste del disco y cada descriptor registra su ajuste. Asignar una partición consume espacio libre existente y conserva el tamaño total del .mia.

4.2 Layout de la partición EXT2

[SuperBlock] -> [Bitmap de inodos] -> [Bitmap de bloques]
-> [Tabla de inodos] -> [Región de bloques de 64 bytes]

Figura 3. Orden de las regiones EXT2 dentro de una partición primaria. El espacio que no alcanza para otro conjunto completo de estructuras queda como remanente.

Sea P el tamaño de la partición, S el tamaño del SuperBlock, I el tamaño del Inode y B = 64 el tamaño del bloque. La ecuación del PDF es $P = S + n + 3n + nI + 3nB$. Por tanto:

$$n = \text{floor}((P - S) / (4 + I + 3B))$$

Total de inodos = n Total de bloques = 3n

Con inicio de partición A, las posiciones se derivan en orden: bitmap de inodos = A + S; bitmap de bloques = A + S + n; tabla de inodos = A + S + 4n; región de bloques = A + S + 4n + nI. El final de esta región debe ser menor o igual que A + P.

El inodo de índice i se lee en s_inode_start + i × I; el bloque de índice j, en s_block_start + j × B. Una referencia lógica es un índice; un desplazamiento físico es una posición en bytes. Mantener esta distinción evita lecturas en regiones equivocadas.

5 Permisos UGO y usuario root

El campo `i_perm` almacena tres caracteres octales. El primero corresponde al propietario (U), el segundo al grupo (G) y el tercero a otros usuarios (O). La categoría se determina comparando la sesión con `i_uid` e `i_gid`: primero propietario, después grupo y finalmente otros.

Valor	Permiso	Significado general
4	Lectura r	Leer contenido o consultar un directorio.
2	Escritura w	Modificar contenido o entradas.
1	Ejecución x	Ejecutar o atravesar, según el tipo de objeto.
$6 = 4 + 2$	rw-	Lectura y escritura.
$7 = 4 + 2 + 1$	rwX	Todos los permisos.

Por ejemplo, 664 concede lectura y escritura al propietario y al grupo, y solo lectura a otros. MKFILE y MKDIR crean objetos con 664 según el PDF. Aunque en un sistema Unix real el acceso a directorios suele requerir ejecución, el enunciado de estos comandos fija 664 y exige escritura sobre la carpeta padre. La simulación debe explicarse con esas reglas específicas, sin sustituirlas por valores convencionales de otro sistema.

5.1 Privilegios de root

Root tiene permisos efectivos 777 sobre todos los archivos y carpetas, independientemente de `i_perm`. Esto no significa que todos los inodos deban almacenar 777: significa que la comprobación de autorización reconoce sus privilegios. Root no queda exento de errores como una ruta inexistente, parámetros inválidos o falta de espacio.

Solo root administra grupos y usuarios mediante MKGRP, RMGRP, MKUSR, RMUSR y CHGRP. CAT exige lectura; MKFILE y MKDIR exigen escritura en el padre. Los comandos del sistema de archivos requieren una sesión, salvo MKFS y LOGIN. Los comandos de administración de discos están disponibles desde el inicio de la aplicación.

5.2 Archivo de usuarios y grupos

Cada partición formateada contiene `/users.txt`. Un grupo se representa como GID, G, Grupo; un usuario como UID, U, Grupo, Usuario, Contraseña. Los nombres de grupo, usuario y contraseña tienen un máximo de 10 caracteres. Un identificador 0 representa una baja lógica y los identificadores aumentan al crear registros.

- 1, G, root
- 1, U, root, root, 123

MKFS crea estos registros iniciales y asigna a su inodo UID 1 y GID 1. El archivo utiliza inodo y bloques como cualquier otro archivo de la simulación. LOGIN consulta registros activos; LOGOUT borra la sesión temporal. Cambiar el grupo de un usuario en `/users.txt` no equivale a reescribir automáticamente `i_gid` de todos sus archivos.

6 Referencia de comandos

Los comandos y nombres de parámetros no distinguen mayúsculas y minúsculas. Los parámetros se separan por espacios, pueden aparecer en cualquier orden y usan -parametro=valor; una bandera como -r o -p no recibe valor. Los valores con espacios se escriben entre comillas dobles. Los nombres de usuario y contraseñas conservan la distinción de mayúsculas indicada por sus reglas.

En los ejemplos, /home/user/Disco1.mia es una ruta del sistema anfitrión. /docs/a.txt es una ruta interna del EXT2 simulado. El ID 471A es ilustrativo para un carnet terminado en 47; debe sustituirse por el que devuelva MOUNT y muestre MOUNTED.

6.1 MKDISK

Crea un archivo .mia de tamaño fijo, inicialmente lleno de ceros binarios, y escribe su MBR.

Parámetro	Condición	Uso y valores
-size	Obligatorio	Entero positivo mayor que cero.
-path	Obligatorio	Ruta del .mia; crea carpetas faltantes.
-unit	Opcional	K o M; predeterminado M.
-fit	Opcional	BF, FF o WF; predeterminado FF.

Ejemplo de uso

```
mkdisk -size=30 -unit=M -fit=FF -path=/home/user/Disco1.mia
```

Efecto interno. Reserva físicamente el tamaño solicitado, inicializa el MBR y deja disponibles sus entradas de partición. No crea todavía SuperBlock, inodos ni archivos internos.

6.2 RMDISK

Elimina el archivo que representa el disco. Si no existe, devuelve un error.

Parámetro	Condición	Uso y valores
-path	Obligatorio	Ruta del archivo .mia que se eliminará.

Ejemplo de uso

```
rmdisk -path=/home/user/Disco1.mia
```

Efecto interno. Al borrar el archivo anfitrión se pierde todo su contenido: MBR, particiones, EBR y sistemas de archivos que contenga. No es una baja lógica de una entrada de directorio interno.

Este ejemplo es independiente del flujo de creación: ejecutarlo elimina el disco empleado por los demás ejemplos.

6.3 FDISK

Administra la creación de particiones dentro de un disco existente. Su descripción general menciona eliminación y modificación, pero la tabla específica solo define parámetros para crear; por ello no se documentan opciones de borrado o redimensionamiento no especificadas.

Parámetro	Condición	Uso y valores
-size	Obligatorio al crear	Tamaño positivo mayor que cero.
-path	Obligatorio	Disco existente.
-name	Obligatorio	Nombre único entre particiones del disco.
-unit	Opcional	B, K o M; predeterminado K.
-type	Opcional	P, E o L; predeterminado P.
-fit	Opcional	BF, FF o WF; predeterminado WF.

Ejemplos de uso

```
fdisk -size=10 -unit=M -type=P -path=/home/user/Disco1.mia -name=Primaria1
```

```
fdisk -size=8 -unit=M -type=E -path=/home/user/Disco1.mia -name=Extendida1
```

```
fdisk -size=2 -unit=M -type=L -fit=BF -path=/home/user/Disco1.mia -name=Logical1
```

Efecto interno. Para una primaria se actualiza una entrada Partition del MBR con inicio, tamaño, tipo, ajuste y nombre. Para una extendida se actualiza su entrada y se inicializa un EBR. Una lógica se registra dentro de la extendida y se mantiene el enlace `part_next` de la cadena.

Se valida la existencia del disco, la unicidad del nombre, la disponibilidad de una entrada cuando corresponde y el espacio suficiente. Primarias y extendidas suman como máximo cuatro; solo puede existir una extendida y no se crea una lógica sin ella. La reserva respeta los límites físicos y no incrementa el tamaño del `.mia`.

La comprobación mediante REP MBR permite observar descriptores y EBR. REP DISK permite verificar la distribución, los huecos libres y los porcentajes, evitando confundir una partición declarada con una región físicamente válida.

6.4 MOUNT y MOUNTED

MOUNT

Registra una partición primaria como montada y le asigna un identificador para operaciones posteriores.

Parámetro	Condición	Uso y valores
-path	Obligatorio	Ruta del disco existente.
-name	Obligatorio	Nombre de la partición primaria existente.

Ejemplo de uso

```
mount -path=/home/user/Disco1.mia -name=Primaria1
```

Efecto interno. El registro de montajes en RAM relaciona el ID con el disco y la partición. La sección del comando también pide actualizar estado, correlativo e ID del descriptor Partition; esos datos corresponden a `part_status`, `part_correlative` y `part_id`.

El PDF contiene una contradicción: una observación prohíbe escribir el montaje en disco y la siguiente exige actualizar atributos de la partición. La interpretación funcional es mantener el registro operativo en RAM y actualizar los campos del descriptor exigidos, conservando explícita esta distinción al revisar la implementación.

El ID combina los últimos dos dígitos del carnet, el número de partición montada dentro de ese disco y una letra de disco. Para terminación 47, el primer montaje del disco A sería 471A; el segundo del mismo disco, 472A; y el primero de otro disco, 471B. El correlativo depende del orden de montaje, no de la posición física de la entrada en el MBR.

MOUNTED

Muestra las particiones montadas en memoria; cada registro debe incluir obligatoriamente el ID.

Parámetros: no recibe parámetros.

Ejemplo de uso

```
mounted
```

Efecto interno. Consulta el registro temporal de montajes. No crea particiones ni modifica la distribución EXT2. Su salida permite elegir el ID correcto para MKFS, LOGIN y REP.

Montar identifica y habilita una partición para operar desde la aplicación. Formatear, en cambio, escribe su organización de sistema de archivos. MOUNT no reemplaza a MKFS.

6.5 MKFS y CAT

MKFS

Realiza un formateo completo EXT2 sobre una partición montada y crea /users.txt en la raíz. No requiere una sesión iniciada.

Parámetro	Condición	Uso y valores
-id	Obligatorio	ID existente generado por MOUNT.
-type	Opcional	Solo full; predeterminado full.

Ejemplo de uso

```
mkfs -id=471A -type=full
```

Efecto interno. Inicializa el SuperBlock, los dos bitmaps, la tabla de inodos y la región de bloques según la fórmula de capacidad. Crea el directorio raíz y users.txt con los registros iniciales de root; marca los recursos utilizados y ajusta los contadores libres.

El formateo sustituye el sistema de archivos previo de la partición. No cambia el tamaño del disco ni requiere un parámetro para seleccionar otro sistema: el proyecto usa EXT2.

CAT

Muestra y concatena el contenido de uno o varios archivos internos en el orden en que fueron indicados. Requiere una sesión y permiso de lectura.

Parámetro	Condición	Uso y valores
-fileN	Obligatorio	Una o más rutas: -file1, -file2, etc.

Ejemplo de uso

```
cat -file1=/users.txt -file2=/docs/a.txt
```

Efecto interno. Localiza cada ruta mediante entradas de carpetas e inodos y recorre bloques directos e indirectos para recuperar el contenido. i_s delimita los bytes válidos y i_atime representa la última lectura. No reserva inodos ni bloques nuevos.

Si un archivo no existe o no hay permiso de lectura, informa el error. Los contenidos se separan mediante salto de línea, como establece el ejemplo del PDF.

6.6 LOGIN y LOGOUT

LOGIN

Inicia una sesión en la partición especificada. No admite otra sesión activa simultáneamente.

Parámetro	Condición	Uso y valores
-user	Obligatorio	Usuario existente; distingue mayúsculas.
-pass	Obligatorio	Contraseña; distingue mayúsculas.
-id	Obligatorio	ID de partición montada con sistema de archivos.

Ejemplo de uso

```
login -user=root -pass=123 -id=471A
```

Efecto interno. Lee /users.txt desde sus inodos y bloques, comprueba un registro activo y valida credenciales. Conserva temporalmente la identidad y partición de la sesión para las operaciones posteriores. No crea un usuario nuevo ni otra partición.

Una contraseña incorrecta produce un fallo de autenticación. Un usuario inexistente o la presencia de otra sesión activa también deben generar un error.

LOGOUT

Cierra la sesión actual. Si no existe una sesión activa, informa un error.

Parámetros: no recibe parámetros.

Ejemplo de uso

```
logout
```

Efecto interno. Limpia los datos temporales de la sesión. Conserva /users.txt y los demás archivos del .mia. Cerrar sesión no elimina al usuario ni equivale a borrar o formatear la partición.

Identidad y pertenencia

La sesión permite comparar al usuario con i_uid e i_gid de cada objeto. El ID de montaje identifica una partición; el UID identifica un usuario; el GID identifica un grupo. Son identificadores con funciones distintas y no deben intercambiarse en las validaciones.

Por ejemplo, login -user=root -pass=123 -id=471A usa 471A para localizar el sistema de archivos y luego encuentra el UID 1 de root en su archivo de usuarios. Un usuario con el mismo nombre en otra partición pertenece a otro contexto de datos.

6.7 MKGRP y RMGRP

MKGRP

Crea un grupo en la partición de la sesión. Solo root puede ejecutarlo y el nombre distingue mayúsculas y minúsculas.

Parámetro	Condición	Uso y valores
-name	Obligatorio	Nombre del grupo; máximo 10 caracteres.

Ejemplo de uso

```
mkgrp -name=usuarios
```

Efecto interno. Lee /users.txt, verifica que el grupo no exista como activo y añade un registro G con un nuevo identificador. Al crecer el archivo actualiza su inodo y contenido; si hacen falta bloques, actualiza referencias, bitmap y SuperBlock.

Un registro de ejemplo es 2, G, usuarios. El comando no crea otra clase de estructura binaria para almacenar grupos.

RMGRP

Elimina lógicamente un grupo existente en la partición de la sesión. Solo root puede ejecutarlo.

Parámetro	Condición	Uso y valores
-name	Obligatorio	Nombre del grupo activo que se elimina.

Ejemplo de uso

```
rmgrp -name=usuarios
```

Efecto interno. Cambia a 0 el identificador del registro de grupo en /users.txt. Conserva el registro como evidencia de baja lógica; no borra el inodo del archivo de usuarios ni todos los archivos pertenecientes al grupo.

El registro 2, G, usuarios pasa a 0, G, usuarios. Si se repite la operación sobre un grupo ya eliminado, se informa que no existe como grupo activo. El PDF no define una eliminación en cascada de sus usuarios.

6.8 MKUSR y RMUSR

MKUSR

Crea un usuario en un grupo activo de la partición de la sesión. Su ejecución está reservada a root.

Parámetro	Condición	Uso y valores
-user	Obligatorio	Nombre único de usuario; máximo 10 caracteres.
-pass	Obligatorio	Contraseña; máximo 10 caracteres.
-grp	Obligatorio	Grupo existente; máximo 10 caracteres.

Ejemplo de uso

```
mkusr -user=user1 -pass=usuario -grp=usuarios
```

Efecto interno. Agrega un registro U a /users.txt con un nuevo UID, grupo, usuario y contraseña. Actualiza el tamaño y contenido del archivo y reserva bloques adicionales si son necesarios.

El usuario debe ser único aun cuando se intente registrarlo en otro grupo. La creación no asigna automáticamente un archivo o una carpeta personal.

RMUSR

Elimina lógicamente un usuario existente. Solo root puede ejecutarlo sobre la partición de la sesión.

Parámetro	Condición	Uso y valores
-user	Obligatorio	Nombre del usuario activo que se elimina.

Ejemplo de uso

```
rmusr -user=user1
```

Efecto interno. Escribe 0 en el UID del registro correspondiente de /users.txt. El registro deja de representar un usuario activo; no se eliminan automáticamente los inodos o bloques de los archivos que haya creado.

Por ejemplo, 2, U, usuarios, user1, usuario se convierte en 0, U, usuarios, user1, usuario. El esquema conserva sus cinco campos aunque algunos ejemplos del PDF aparezcan cortados por saltos de línea.

6.9 CHGRP y administración de registros

CHGRP

Cambia el grupo al que pertenece un usuario. Requiere sesión de root, un usuario existente y un grupo de destino activo.

Parámetro	Condición	Uso y valores
-user	Obligatorio	Usuario al que se asignará otro grupo.
-grp	Obligatorio	Nuevo grupo existente y no eliminado.

Ejemplo de uso

```
chgrp -user=user1 -grp=estudio
```

Efecto interno. Modifica el campo Grupo del registro U en /users.txt. Conserva el UID, nombre y contraseña. Si cambia la longitud del contenido, ajusta el archivo con sus mecanismos normales de escritura.

Antes del ejemplo debe existir el grupo estudio, que puede crearse con `mkgrp -name=estudio`. CHGRP no es un comando para cambiar directamente el grupo propietario de un archivo.

Secuencia de actualización de users txt

Los comandos de administración localizan /users.txt mediante el sistema de archivos, interpretan los registros, validan la operación y escriben el contenido actualizado. Si el archivo supera un bloque, sus registros pueden distribuirse entre varios FileBlock; la lectura lógica debe reconstruirlos en el orden correcto.

El archivo de usuarios no debe mantenerse como una copia permanente en memoria que sustituya al .mia. Los datos persistidos deben ser la referencia para las operaciones y para los reportes. El formato de grupos y usuarios se conserva al reescribir; las bajas lógicas se distinguen de las entradas ocupadas de los bitmaps.

Para comprobar estos comandos puede ejecutarse `cat -file1=/users.txt` con una sesión autorizada o generar REP FILE. Una baja lógica en un registro de usuario no pone automáticamente a cero el bitmap del inodo de /users.txt: ese archivo sigue existiendo y sigue ocupado.

6.10 MKFILE

Crea un archivo en la partición de la sesión. El propietario es el usuario actual, sus permisos iniciales son 664 y debe existir permiso de escritura sobre la carpeta padre.

Parámetro	Condición	Uso y valores
-path	Obligatorio	Ruta interna del archivo.
-r	Opcional	Crea padres faltantes; no recibe valor.
-size	Opcional	Bytes no negativos; predeterminado 0.
-cont	Opcional	Ruta de un archivo del anfitrión cuyo contenido se importa; prevalece sobre -size.

Ejemplos de uso

```
mkfile -path=/docs/a.txt -size=15 -r
```

```
mkfile -path=/docs/b.txt -cont=/home/user/contenido.txt -r
```

El primer ejemplo produce 012345678901234. Con -size, el contenido repite los dígitos del 0 al 9 hasta alcanzar el tamaño solicitado. Sin -size ni -cont, el archivo tiene 0 bytes. Un tamaño negativo o un archivo anfitrión inexistente para -cont produce un error.

Efecto interno. Se resuelve la carpeta padre y, con -r, se crean los padres faltantes. Se asigna el inodo del archivo con UID, GID, permisos, fechas, tipo y tamaño. Una entrada Content de la carpeta padre relaciona el nombre con ese inodo. Se reservan FileBlock y, cuando corresponde, PointerBlock; se actualizan bitmaps y contadores libres.

Si el archivo ya existe, la aplicación debe preguntar si se desea sobrescribirlo. La sustitución del contenido debe mantener consistentes `i_s`, las referencias a bloques y la ocupación registrada. Un archivo vacío necesita representación mediante inodo y entrada de carpeta, aunque no tenga bytes de contenido.

El parámetro -cont hace referencia a un archivo accesible para el proceso del backend, no a una ruta interna del .mia. En cambio, -path determina dónde se crea el archivo simulado. Esta distinción permite explicar por qué dos rutas del mismo comando pertenecen a sistemas de almacenamiento distintos.

6.11 MKDIR y REP

MKDIR

Crea una carpeta interna con propietario igual al usuario de la sesión y permisos 664. Exige escritura en la carpeta padre.

Parámetro	Condición	Uso y valores
-path	Obligatorio	Ruta de la carpeta.
-p	Opcional	Crea padres faltantes; no recibe valor.

Ejemplo de uso

```
mkdir -p -path=/home/user/docs
```

Efecto interno. Asigna un inodo de carpeta con `i_type = 0`, sus bloques de directorio y la entrada en el padre. Inicializa los registros que relacionan la carpeta consigo misma y con su padre, y actualiza bitmaps y SuperBlock.

La descripción de `-path` menciona `-r` por error, pero la tabla y los ejemplos de MKDIR establecen `-p`. En este manual se utiliza `-p`; `-r` corresponde a MKFILE.

REP

Genera uno de los diez reportes definidos por el proyecto a partir del disco o de la partición indicada.

Parámetro	Condición	Uso y valores
-name	Obligatorio	mbr, disk, inode, block, bm_inode, bm_block, tree, sb, file o ls.
-path	Obligatorio	Ruta de salida en el anfitrión; crea carpetas faltantes.
-id	Obligatorio	ID de la partición montada.
-path_file_ls	Condicional	Ruta interna para FILE y LS; opcional en la tabla general.

Ejemplo de uso

```
rep -name=tree -path=/home/user/reports/tree.jpg -id=471A
```

Efecto interno. Lee las estructuras y produce una representación externa. Para reportes de disco localiza el `.mia` desde el ID; para reportes EXT2 consulta la partición. No reserva objetos del sistema simulado. La sección 8 detalla contenido y ejemplos de todos los reportes.

7 Scripts smia y comentarios

Un script .smia es un archivo de texto con comandos del proyecto. Se carga en el área de entrada del frontend y se ejecuta en orden, de arriba hacia abajo. Conviene escribir un comando completo por línea; los saltos causados por la maquetación de los ejemplos del PDF no forman parte de su sintaxis.

Los comentarios ocupan una línea y comienzan con #. Se muestran tal como aparecen en el script; no ejecutan operaciones. Las líneas en blanco también se preservan. Los comentarios permiten dividir una carga masiva por etapas y explicar qué se espera de cada comando.

7.1 Ejemplo de carga organizada

```
# Preparación del disco
mkdisk -size=30 -unit=M -path=/home/user/Disco1.mia
fdisk -size=10 -unit=M -path=/home/user/Disco1.mia -name=Primaria1
mount -path=/home/user/Disco1.mia -name=Primaria1
mounted

# Formateo e inicio de sesión
mkfs -id=471A
login -user=root -pass=123 -id=471A

# Creación y lectura
mkgrp -name=usuarios
mkusr -user=user1 -pass=usuario -grp=usuarios
mkdir -path=/docs -p
mkfile -path=/docs/a.txt -size=15
cat -file1=/docs/a.txt

# Verificación y cierre
rep -name=tree -path=/home/user/reports/tree.jpg -id=471A
logout
```

El ejemplo supone un disco nuevo y que el primer montaje devuelve 471A. Si MOUNT devuelve otro identificador, debe reemplazarse antes de ejecutar los comandos dependientes. Para un primer uso puede ejecutarse la preparación y consultar MOUNTED antes de continuar.

7.2 Reglas de interpretación

No se agregan parámetros desconocidos ni valores a las banderas. Las rutas con espacios se encierran entre comillas dobles. La lectura del script debe conservar los comentarios y líneas en blanco para relacionar la salida con cada sección. La carga del archivo incorpora su texto a la interfaz; la ejecución es la acción que solicita procesarlo al backend.

8 Reportes del sistema

Los reportes evidencian el estado persistido. Los gráficos se generan mediante Graphviz según el apartado de reportes; los bitmaps se exportan en texto con 20 registros por línea. FILE también tiene un ejemplo de salida .txt en el PDF. Los tamaños reducidos de algunas figuras del enunciado son ilustrativos y no cambian las capacidades reales de las estructuras.

8.1 MBR y DISK

Reporte	Información que debe mostrar	Comprobación principal
MBR	Todos los atributos del MBR y de los EBR creados.	Descriptores, nombres, tamaños y enlaces de lógicas.
DISK	MBR, particiones y espacios libres con porcentajes.	Distribución física sin solapamientos y cobertura del disco.

```
rep -name=mbr -path=/home/user/reports/mbr.jpg -id=471A
```

```
rep -name=disk -path=/home/user/reports/disk.pdf -id=471A
```

En DISK, el porcentaje de una región respecto del disco se calcula como tamaño de la región / $mbr_tamano \times 100$. La suma de las regiones no superpuestas debe ser 100 %, salvo diferencias de redondeo. La extendida ya contiene sus lógicas: no se suman ambas nuevamente como regiones independientes del disco completo.

8.2 INODE y BLOCK

Reporte	Información que debe mostrar	Comprobación principal
INODE	Todos los campos de los inodos utilizados.	Propietarios, tamaños, tipos, permisos y referencias.
BLOCK	Información de los bloques utilizados y su interpretación.	Entradas de carpeta, contenido y enlaces de apuntadores.

```
rep -name=inode -path=/home/user/reports/inode.jpg -id=471A
```

```
rep -name=block -path=/home/user/reports/block.jpg -id=471A
```

Los recursos libres no se muestran como objetos usados. El bitmap permite identificar ocupación; los inodos y sus referencias permiten conocer el tipo y relación de los bloques. Estos reportes ayudan a comprobar MKFS y la asignación realizada por MKFILE y MKDIR.

8.3 Bitmaps TREE y SB

bm_inode y bm_block

bm_inode presenta todas las entradas del bitmap de inodos y bm_block las del bitmap de bloques. Ambos incluyen libres y ocupadas, utilizan salida de texto y colocan 20 registros por línea; la última puede contener menos si el total no es múltiplo de 20.

```
rep -name=bm_inode -path=/home/user/reports/bm_inode.txt -id=471A
```

```
rep -name=bm_block -path=/home/user/reports/bm_block.txt -id=471A
```

La tabla de REP y la rúbrica denominan el reporte bm_block. Un encabezado y un ejemplo posteriores escriben bm_bloc; se usa bm_block para conservar el nombre del catálogo de parámetros. Contar ceros y unos permite comparar ocupación con los contadores del SuperBlock.

TREE

Representa el árbol completo del EXT2 mostrando inodos, bloques y enlaces. Debe permitir seguir una carpeta hacia sus entradas, de cada entrada al inodo correspondiente y de un archivo a su contenido, incluyendo los niveles de apuntadores utilizados.

```
rep -name=tree -path=/home/user/reports/tree.jpg -id=471A
```

Al interpretar TREE, las referencias a la propia carpeta y al padre no representan nuevos objetos que deban recorrerse indefinidamente. El reporte debe distinguir esas relaciones del descenso hacia los hijos. Es la evidencia principal de que una ruta creada tiene inodo, entrada de directorio y bloques coherentes.

SB

Muestra todos los campos del SuperBlock en una tabla: tipo de sistema, capacidades, contadores libres, fechas, cantidad de montajes, marca 0xEF53, tamaños, primeras posiciones libres e inicios de regiones.

```
rep -name=sb -path=/home/user/reports/sb.jpg -id=471A
```

SB permite revisar que el total de bloques sea triple del de inodos y que los límites de las regiones respeten la partición. Debe contrastarse con los bitmaps: un contador de bloques libres que no coincide con sus ceros revela inconsistencia en el registro del espacio.

8.4 FILE y LS

FILE

Muestra el nombre y todo el contenido del archivo interno indicado por `-path_file_ls`. La ruta debe existir en la partición asociada con `-id`. El contenido se reconstruye siguiendo el inodo y sus bloques, sin confundir la ruta interna con el destino anfitrión de `-path`.

```
rep -name=file -path=/home/user/reports/users.txt -id=471A -path_file_ls=/users.txt
```

Es útil para revisar altas, bajas y cambios de grupo. El nombre del parámetro operativo es `-path_file_ls`; la mención genérica a “file” en la descripción no introduce un parámetro adicional. La tabla lo clasifica como opcional globalmente porque los otros reportes no lo necesitan; para seleccionar el objeto de FILE y LS se suministra la ruta.

LS

Muestra información de archivos y carpetas: permisos, propietario, grupo propietario, fecha y hora de modificación, tipo y fecha de creación. Las entradas del directorio proporcionan nombres; los inodos proporcionan metadatos; los registros de usuarios y grupos permiten interpretar los identificadores de propietario.

```
rep -name=ls -path=/home/user/reports/ls.jpg -id=471A -path_file_ls=
```

8.5 Relación entre operaciones y evidencias

Operación	Evidencia apropiada
MKDISK y FDISK	MBR y DISK para descriptores y distribución.
MOUNT	MOUNTED para el registro de IDs.
MKFS	SB, bitmaps, INODE, BLOCK o TREE para inicialización.
Usuarios y grupos	CAT sobre /users.txt o reporte FILE.
MKFILE y MKDIR	TREE para enlaces; INODE y BLOCK para detalles.
Permisos y propiedad	LS e INODE para atributos; resultado del comando para autorización.

Una figura correcta debe reflejar lecturas del disco y no datos preparados solo para la presentación. La verificación técnica compara el comando ejecutado, la salida devuelta y las estructuras mostradas. Un reporte complementa la comprobación de permisos; mostrar 664 no demuestra por sí solo que se haya aplicado la validación.